

ARCHITECTURE REPORT

1. REQUIREMENTS

```
{
  "system_name": "OnlineFoodOrderingSystem",
  "architecture_style": "Event-Driven Microservices Architecture",
  "functional_requirements": [
    "User registration and authentication",
    "Browse restaurants and menus",
    "Place food orders",
    "Process online payments",
    "Track order status in real time",
    "Send order notifications to users"
  ],
  "non_functional_requirements": {
    "scalability": "High",
    "availability": "High",
    "latency": "Medium",
    "reliability": "High",
    "consistency": "Eventual"
  }
}
```

2. ARCHITECTURE (ACME ADL)

```
System OnlineFoodOrderingSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Event-Driven Microservices Architecture";
  Property source : String = "AI";
  Component UserService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Handles user registration, authentication, and profile management.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component RestaurantService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Manages restaurant information, menus, and order processing.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component OrderService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Processes food orders, handles payment, and tracks order status in real time.";
  }
}
```

```

    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component PaymentGateway = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Facilitates online payments through various payment methods.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
    Property type : String = "L7";
    Property healthChecks : Boolean = true;
}
Component NotificationService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Sends order notifications to users via email, SMS, or other channels";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Database = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Stores and retrieves data for all components.";
    Property state : String = "stateful";
    Property scaling : String = "vertical";
    Property consistency : String = "eventual";
    Property replication : String = "multi-node";
}
Component APIGateway = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Routes incoming requests to the respective components and handles errors";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
    Property type : String = "L7";
    Property healthChecks : Boolean = true;
}
Component WebApplication = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Provides a user-friendly interface for users to interact with the system";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Attachment APIGateway.out to UserService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment APIGateway.out to RestaurantService.in {

```

```

    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment APIGateway.out to OrderService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment APIGateway.out to PaymentGateway.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment APIGateway.out to NotificationService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment UserService.out to Database.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment RestaurantService.out to Database.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment OrderService.out to Database.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment PaymentGateway.out to Database.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment NotificationService.out to Database.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment OrderService.out to NotificationService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment PaymentGateway.out to NotificationService.in {

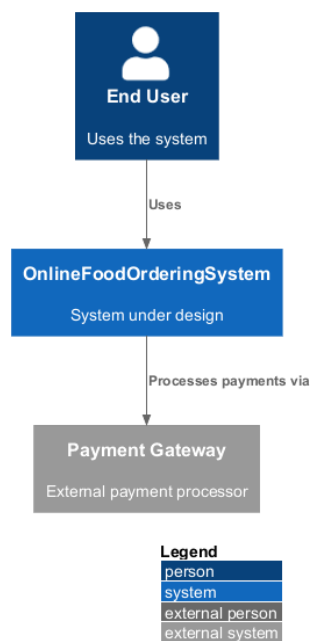
```

```

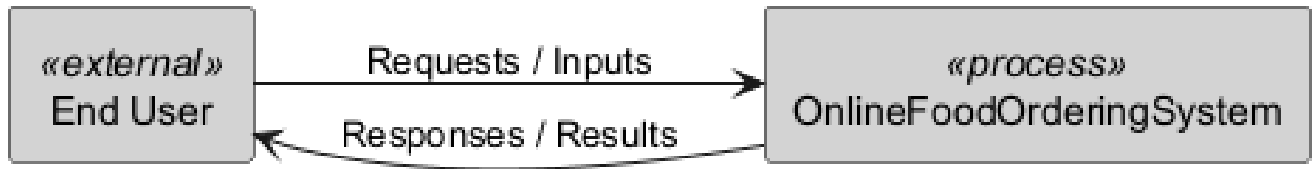
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property backpressure : String = "bounded";
}
Attachment WebApplication.out to APIGateway.in {
Property delivery : String = "at-least-once";
Property ordering : Boolean = true;
Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";
}

```

3. CONTEXT VIEW



3.1 DATA CONTEXT VIEW (LEVEL 0 DFD)



4. C4 CONTAINER DIAGRAM

